

Міністерство освіти і науки України
Львівський Національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
з курсу «Хмарні технології та великі дані»
«Circuit Breaker (Розмикач ланцюжка) »

Виконав:
Студент групи ФеІ-43
Ярема Р.Н.

Перевірив:
асистент Керод Т.І.

Львів 2026

Мета роботи: запобігти каскадним відмовам при збоях віддалених сервісів, швидко «відсікати» несправний шлях виклику.

Git ref: <https://github.com/Roman5917/cloud.git>

Хід роботи

У ході лабораторної роботи було реалізовано шаблон Circuit Breaker мовою JavaScript. Його основне призначення - контролювати виклики до зовнішнього сервісу та тимчасово блокувати їх у випадку великої кількості помилок.

Circuit Breaker має три основні стани: **Closed**, **Open** та **HalfOpen**. Початковим станом є Closed, у якому всі виклики дозволені. Для роботи розмикача задаються кількість допустимих помилок `failureThreshold`, максимальна кількість перевірочних викликів `halfOpenMaxCalls`, тривалість стану Open `openStateDuration` та максимальний час виконання одного виклику `timeoutPerCall`.

Основними методами класу є `call(fn)` та `state()`. Метод `state()` повертає поточний стан Circuit Breaker, а `call(fn)` виконує передану функцію та контролює результат її виконання.

```
state() {  
    return this.currentState;  
}  
  
async call(fn) {  
    // виконання та контроль переданої функції  
}
```

Якщо під час роботи у стані Closed кількість послідовних помилок досягає заданого значення `failureThreshold`, Circuit Breaker переходить у стан Open та запам'ятовує час переходу. У цьому стані нові виклики блокуються.

```
this.failureCount++;
```

```
if (this.failureCount >= this.failureThreshold) {  
    this.currentState = "Open";  
    this.openedAt = Date.now();  
}
```

```
}
```

Після завершення `openStateDuration` розмикач переходить у стан `HalfOpen`, у якому дозволяється обмежена кількість пробних викликів. Якщо вони виконуються успішно, система повертається до `Closed`. Якщо виникає помилка - знову переходить у `Open`.

Для контролю часу виконання операції використовується `Promise.race()`.

Одночасно запускається передана функція та таймер. Якщо функція не завершується за `timeoutPerCall`, генерується помилка `Timeout`.

```
const result = await Promise.race([
  fn(),
  new Promise((_, reject) => {
    setTimeout(() => {
      reject(new Error("Timeout"));
    }, this.timeoutPerCall);
  })
]);
```

Для перевірки роботи програми створено тестові функції, які імітують успішну роботу сервісу, виникнення помилки та тривале очікування відповіді.

Під час виконання програми було перевірено перехід зі стану `Closed` у `Open` після трьох помилок, блокування наступного виклику, очікування 5 секунд та подальший перехід у `HalfOpen`. Після двох успішних перевірочних викликів `Circuit Breaker` повертається до стану `Closed`.

Висновок

Під час виконання лабораторної роботи було реалізовано та досліджено роботу шаблону `Circuit Breaker`. Реалізовано контроль помилок і тайм-аутів, блокування викликів при недоступності сервісу та автоматичне відновлення роботи після успішних перевірочних викликів.